



CCF-GESP C++四级

编程能力等级认证



第十五课

时间空间复杂度



01 时间复杂度



时间复杂度



算法在执行时**需要消耗的时间**。

使用某些**固定操作**被执行的**次数**，和**n的关系**来进行度量。

```
int n=10;  
int sum=0;  
sum+=n; 固定操作  
sum+=n;  
sum+=n;
```

- 执行次数与变量n无关
- 时间复杂度 **$O(1)$**
- 常量级



时间复杂度



算法在执行时**需要消耗的时间**。

使用某些**固定操作**被执行的**次数**，和**n的关系**来进行度量。

```
int n=10;  
int sum=0;  
for(int i=0;i<n;i++){  
    sum+=i;  
}
```

n次

固定操作

- 执行n次
- 时间复杂度 $O(n)$
- 线性级



知识讲解

时间复杂度



算法在执行时**需要消耗的时间**。

使用某些**固定操作**被执行的**次数**，和**n的关系**来进行度量。

$n \times n$ 次

```
int a[100][100]={0};  
int n=10;  
for(int i=1;i<=n;i++){  
    for(int j=1;j<=n;j++){  
        cout<<a[i][j]<<' '  
        cout<<endl;  
    }  
}
```

n次

n次

- 执行 n^2 次
- 时间复杂度 $O(n^2)$



时间复杂度



算法在执行时**需要消耗的时间**。

使用某些**固定操作**被执行的**次数**，和**n的关系**来进行度量。

```
int a[100][100]={0};  
int n=10;  
for(int i=0;i<n;i++){  
    for(int j=0;j<n;j++){  
        cout<<a[i][j]<<' ';  
        cout<<endl;  
    }  
int sum=0;  
for(int i=0;i<n;i++){  
    sum+=i;  
}
```

$n \times n$ 次

n 次

- 关注随n的增长而增长得**更快**的那一项操作，时间复杂度 **$O(n^2)$**



知识讲解

时间复杂度



算法在执行时**需要消耗的时间**。

使用某些**固定操作**被执行的**次数**，和**n的关系**来进行度量。

```
int i = 1;  
int n=10;  
while(i<n){  
    i*=2;  
}
```

$2^x =$
 n

- 执行 $\log_2 n$ 次
- 时间复杂度 $O(\log_2 n)$ 或者 $O(\log n)$
- 对数级



其他的时间复杂度还有 $O(n \log n)$ 、 $O(n^3)$ 、 $O(2^n)$ 、 $O(n!)$



时间复杂度的大小关系



哪种时间更长，哪种更节省时间呢？
按照所消耗时间由少到多进行排序如下：

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$



试一试

```
long long cnt=0;  
int n=100000;  
for(int i=1;i<=n;i++){  
    for(int j=1;j<=n;j++)  
        cnt++;  
}
```

$100000 \times 100000 \rightarrow 100$
亿次



一个程序运算次数高达100亿次，提交任何一个平台都会超时！！！！



知识讲解

排序算法时间复杂度

冒泡排序

```
for(int i=0;i<n-1;i++){  
    bool flag=false;  
    for(int j=0;j<n-i;j++){  
        if(a[j]>a[j+1]){  
            swap(a[j],a[j+1]);  
            flag=true;  
        }  
    }  
    if(!flag) break;  
}
```

n-1
次

n-i
次

$(n-1)+(n-2)+\dots+1$

$\frac{n(n-1)}{2}$

平均时间复杂度 $O(n^2)$

最优情况是初始序列**有序**，则时间复杂度为 $O(n)$

选择排序

```
int t,min=1000;  
for(int i=0;i<n-1;i++){  
    t=i;  
    min=a[i];  
    for(int j=i+1;j<n;j++){  
        if(a[j]<min){  
            min=a[j];  
            t=j;}}  
    swap(a[t],a[i]);  
}
```

n-1
次

n-i-1
次

平均时间复杂度 $O(n^2)$



排序算法时间复杂度

插入排序

```
for(int i=1;i<n;i++){  
    int x=a[i];  
    int j;  
    for(j=i-1;j>=0;j--){  
        if(a[j]>x)a[j+1]=a[j];  
        else break;  
    }  
    a[j+1]=x;  
}
```

n-1次

i-1次

平均时间复杂度 $O(n^2)$

排序算法	平均时间复杂度
冒泡排序	$O(n^2)$
选择排序	$O(n^2)$
插入排序	$O(n^2)$



知识讲解

排序算法时间复杂度

桶排序

```
int a[1000]={};  
for(int i=0;i<n;i++) {  
    cin>>x;  
    a[x]++;  
}
```

n次

平均时间复杂度 $O(n)$

排序算法	平均时间复杂度
冒泡排序	$O(n^2)$
选择排序	$O(n^2)$
插入排序	$O(n^2)$
桶排序	$O(n)$



选择题

(2023年9月) 对包含 n 个元素的数组进行冒泡排序，平均时间复杂度一般为 (**C**)

- A. $O(n)$
- B. $O(n \log n)$
- C. $O(n^2)$
- D. 以上都不正确



判断题

(2023年9月) 对N个元素的数组执行插入排序算法，通常的时间复杂度为 $O(N^2)$ 。

☒ 正确

错误 ☐



真题解析



判断题

(2023年12月) 用递归法求的 n 阶乘, 时间复杂度是 $O(n)$ 。

☒ 正确

错误 ☐

选择题

在对 n 个元素进行冒泡排序的过程中，最好情况下的时间复杂度为（ **D** ）

- A. $O(1)$
- B. $O(\log_2 n)$
- C. $O(n^2)$
- D. $O(n)$



02 空间复杂度



空间复杂度



算法在运行过程中**所需的存储空间**

空间复杂度 $O(1)$ ：当一个算法的空间复杂度为一个**常量**，运行时所需空间不随被处理数据量的大小而改变。



知识讲解



排序算法空间复杂度

冒泡排序

```
for(int i=0;i<n-1;i++){  
    bool flag=false;  
    for(int j=0;j<n-i;j++){  
        if(a[j]>a[j+1]){  
            swap(a[j],a[j+1]);  
            flag=true;  
        }  
    }  
    if(!flag) break;  
}
```

平均空间复杂度 **$O(1)$**

选择排序

```
int t,min=1000;  
for(int i=0;i<n-1;i++){  
    t=i;  
    min=a[i];  
    for(int j=i+1;j<n;j++){  
        if(a[j]<min){  
            min=a[j];  
            t=j;}}  
    swap(a[t],a[i]);}
```

平均空间复杂度 **$O(1)$**



排序算法空间复杂度

插入排序

```
int x;  
for(int i=0;i<n;i++){  
    int x=a[i];  
    int j;  
    for(j=i-1;j>=0;j--){  
        if(a[j]>x) a[j+1]=a[j];  
        else break;  
    }  
    a[j+1]=x;  
}
```

平均空间复杂度 $O(1)$

排序算法	空间复杂度
冒泡排序	$O(1)$
选择排序	$O(1)$
插入排序	$O(1)$



排序算法空间复杂度

桶排序

```
int a[1000]={};  
for(int i=0;i<n;i++) {  
    cin>>x;  
    a[x]++;  
}
```

额外申
请空间

平均空间复杂度 $O(n)$

排序算法	空间复杂度
冒泡排序	$O(1)$
选择排序	$O(1)$
插入排序	$O(1)$
桶排序	$O(n)$



排序算法时间空间复杂度

- 一般情况下，空间复杂度没有时间复杂度重要，只要不是创建多个大容量的数组，存储空间都是够用。
- 在解决问题时常常会采用**空间换时间**的策略，例如桶排序。

排序算法	平均时间复杂度	空间复杂度
冒泡排序	$O(n^2)$	$O(1)$
选择排序	$O(n^2)$	$O(1)$
插入排序	$O(n^2)$	$O(1)$
桶排序	$O(n)$	$O(n)$

判断题

插入排序平均时间复杂度为 $O(n^2)$ ，空间复杂度为1。

☒ 正确

错误 ☐

选择题

在对 n 个元素进行直接插入排序的过程中，算法的空间复杂度为（ **A** ）

- A. $O(1)$
- B. $O(\log_2 n)$
- C. $O(n^2)$
- D. $O(n)$

信息老师的前缀和

【问题描述】

运用前缀和思想进行数据预处理，能够避免数据的重复计算，且大大降低了算法的时间复杂度。这天，大白老师想考一考大家对前缀和的掌握情况，于是出了一道题：求 $a \sim b$ 之间1出现的次数。

【输入描述】

共 $n+1$ 行；

第一行，一个整数 n ($1 \leq n \leq 10^5$)，表示查询次数。

接下来 n 行，每行包含两个整数 a 和 b ($1 \leq a < b \leq 10^5$)，数与数之间以一个空格隔开。

【输出描述】

n 行，每行一个整数，分别表示每次循环时1出现的次数。

信息老师的前缀和

【输入样例】

3

1 10

20 30

100 110

【输出样例】

2

1

13

1~10: 1出现的次数: 2

1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

20~30: 1出现的次数: 1

20	21	22	23	24	25	26	27	28	29	30
----	----	----	----	----	----	----	----	----	----	----

100~110: 1出现的次数: 13

100	101	102	103	104	105	106	107	108	109	110
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----



课堂练习

预处理

预先算出1~100000每个数有多少个1:

1~100000:

0	1	2	3	4	5	6	...	10	11	12	13	14	15	..	101	102	...
														.			

数组a保存每个数字中1出现的次数:

0	1	0	0	0	0	0	...	1	2	1	1	1	1	..	2	1	...
														.			

1

7

3

计算1~5之间1出现的次数: 1

计算10~15之间1出现的次数: 7

计算101~102之间1出现的次数: 3

预处理

数组a保存每个数字中1出现的次数:

0	1	0	0	0	0	0	0	0	0	1	2	...
0	1	2	3	4	5	6	7	8	9	10		

11
...
使用前缀和数组sum:

0	1	1	1	1	1	1	1	1	1	2	4	...
0	1	2	3	4	5	6	7	8	9	10		

11
...
sum[1] = sum[1]
sum[2] = sum[2] + sum[1]
sum[3] = sum[3] + sum[2] + sum[1]
.....

sum[3] 表示1~3范围内，1累计出现的次数
sum[6] 表示1~6范围内，1累计出现的次数

sum[6]-sum[3] -> 4~6范围内，1累计出现的次数



思路分析

① 预先算出1~100000每个数有多少个1。

```
int sum[100005]={0};  
for(int i=1;i<=100000;i++){  
    //求当前数字有多少个1  
    int cnt=func(i);  
    sum[i]=sum[i-1]+cnt;  
    前缀和数组  
}
```

```
//func函数求当前数字有多少个1  
int func(int x){  
    int cnt=0;  
    while(x!=0){  
        if(x%10==1)cnt++;  
        x/=10;  
    }  
    return cnt;  
}
```


思路分析

- ② 输入查询次数n，每次查询的区间。
- ③ 一边输入，一边计算本次查询的区间范围内1出现的次数。

```
int n;  
cin>>n;  
int a,b;  
for(int i=0;i<n;i++){  
    cin>>a>>b;  
    cout<<sum[b]-sum[a-1]<<endl;  
}
```

1~a-1范围内1出现的次数

1~b范围内1出现的次数

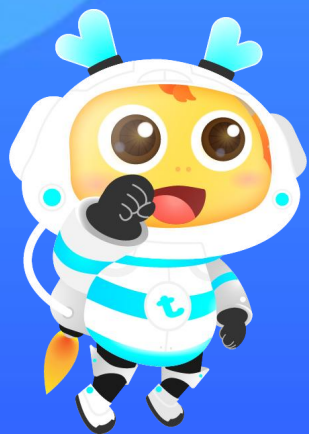


课堂练习

【完整代码】

```
#include <bits/stdc++.h>
using namespace std;
int sum[100005];
int func(int x){
    int cnt=0;
    while(x!=0){
        if(x%10==1)cnt++;
        x/=10;
    }
    return cnt;
}
```

```
int main(){
    for(int i=1;i<=100000;i++){
        int cnt=func(i);
        sum[i]=sum[i-1]+cnt;
    }
    int n;
    cin>>n;
    int a,b;
    for(int i=0;i<n;i++){
        cin>>a>>b;
        cout<<sum[b]-sum[a-1]<<endl;
    }
    return 0;
}
```



感谢你的学习